



youseNDit™

PHP Library Guide
Version 2.5.0

June 28, 2012

Contents

1. Setup.....	3
1.1. Prerequisites.....	3
1.2. Installation and usage	3
2. Library Usage	5
3. Sample App Usage	6
4. Library Reference.....	7
4.1. Authentication.php.....	7
4.1.1. Login	7
4.2. UserAPIs.php.....	8
4.2.1. Create user	8
4.2.2. Get user info	9
4.2.3. Get policy info.....	9
4.3. SendReceive.php	11
4.3.1. Get upload URL for a new item	11
4.3.2. Commit item send	12
4.3.3. Get upload status	12
4.3.4. Get list of items in user's Inbox and/or Sent items	13
4.3.5. Get item count	14
4.3.6. Get item info.....	15
4.3.7. Change expiration of an item in user's Sent Items.....	16
4.3.8. Delete an item from user's Inbox or Sent Items.....	17
4.4. StorageAPIs.php	18
4.4.1. Create new folder	18
4.4.2. Get folder info	19
4.4.3. Get file info	19
4.4.5. Commit file upload to folder	21
4.4.6. Rename a file in a folder	22
4.4.7. Move a file to another folder	23
4.4.8. Delete a file from a folder	23
4.4.9. Rename folder	24
4.4.10. Move folder	25
4.4.11. Delete a folder.....	25
4.4.12. Get file revisions.....	26
4.4.13. Get Storage Revisions.....	27
4.5. FolderShareSendAPIs.php.....	28
4.5.1. Share a folder.....	28
4.5.2. Get share info	28
4.5.3. Unshare a folder	29
4.5.4. Send a file from a folder.....	30
4.6. FileTransfer.php.....	32
4.6.1. Upload File	32
4.6.2. Download File	32
4.6.3. Download File Without Saving on the PHP server	33
4.6.4. Download File with Resume Support	34

1. Setup

1.1. Prerequisites

Apache Server

While developing and testing the application we used XAMPP which contains the Apache server and other tools. You can download and install XAMPP from the following link:

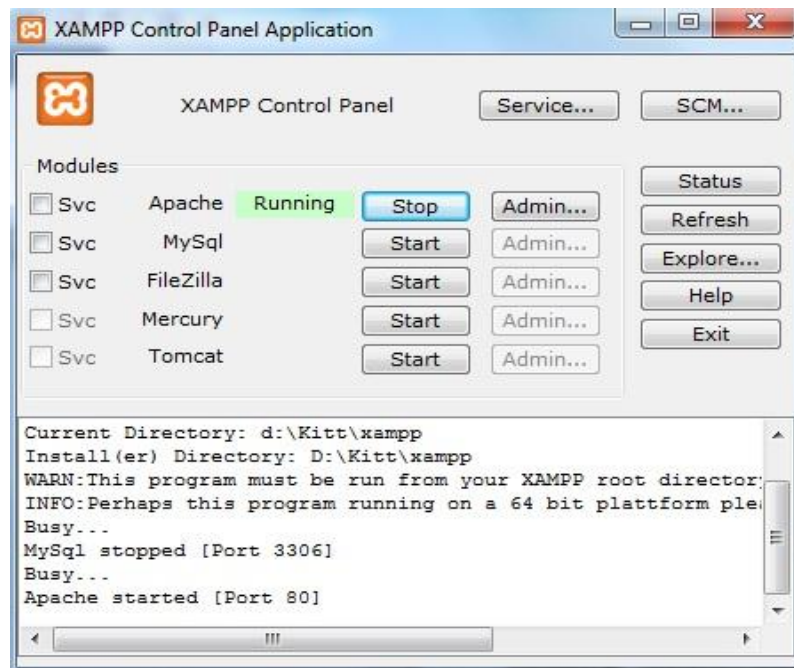
<http://www.apachefriends.org/en/xampp-windows.html#641>

The installer package is recommended for its ease of use.

NOTE: Please remember the installation path, hereafter referred to as YSI_PATH, as it will be needed when installing the YouSendIt Sample application.

1.2. Installation and usage

- Unzip the YouSendIt package
- Copy the sample folder to YSI_PATH\htdocs
- Some parameters need to be changed for the application to work properly:
 - Open YSI_PATH\php\php.ini with a text editor, e.g. Notepad
 - Search for “*extension=php_curl.dll*” and uncomment it by deleting the semi-colon (;) from the start of the line
 - Search for “*display_errors = On*” and modify it to read “*display_errors = Off*”
 - Search for “*error_reporting = E_ALL | E_STRICT*” and change it to “*error_reporting = E_ALL & ~E_WARNING*”
 - Save the file
 - Restart the Apache server.
- If XAMPP is not already started, you can start it by launching YSI_PATH\xampp_control.exe
- The application panel should appear with a control panel similar to the following:



- Press the **Start** button next to Apache

The application is now ready to use. To access the start page just access the link below:

<http://localhost/sample/SampleApp.php>

The Sample App runs on latest browser versions (Chrome, Mozilla and IE). For earlier versions of IE, like IE 8 or IE 7, the display might become corrupted however the functionality is not affected.

2. Library Usage

The PHP Library is delivered in the following folder: “*lib\src*” and consists of a series of classes (.php files). Each class contains a certain number of methods which are wrappers for the REST APIs mentioned in the YSI API Developer’s Guide. Listed below are the PHP classes and their corresponding methods:

- Authentication.php
 - login
- FileTransfer.php
 - uploadaFile
 - downloadaFile
 - downloadaFileWS
 - downloadaFileResume
- FolderShareSendAPIs.php
 - shareFolder
 - getShareInfo
 - unshareFolder
 - sendFile
- SendReceive.php
 - getUploadURLs
 - commitSend
 - getUploadStatus
 - getItemsList
 - getItemsCount
 - getItemInfo
 - changeExpiration
 - deleteItem
 - getDownloadUrl
- StorageAPIs.php
 - createFolder
 - getFolderInfo
 - getFileInfo
 - initUpload
 - commitFileUpload
 - renameFile
 - moveFile
 - deleteFile
 - renameFolder
 - moveFolder
 - deleteFolder
 - getStorageRevisions
 - getFileRevisions
- UserAPIs.php
 - createUser
 - getUserInfo
 - getPolicyInfo

In order to use the PHP Library, you must include the classes containing the desired methods. This can be done by adding the following statement to the beginning of a script:

```
require_once('<relativepath>/<filename.php>');
```

For example, the following statement makes all the methods in the *StorageAPI* class available for use:

```
require_once('StorageAPIs.php');
```

3. Sample App Usage

The PHP Sample application is a client-server-server implementation:

- The client, e.g., a browser, communicates with the PHP Apache Server.
- The PHP Apache server acts as middleware between the client and the YouSendIt Servers.

The PHP Apache server operates independently of the client, and can be accessed from any location. It translates client's requests into REST calls and executes these calls on the YouSendIt server. In the same way, responses received from YouSendIt will be forwarded from the PHP Apache server to the client.

- The client application runs in a browser and supports the latest browser versions
- The browser caching functionality stores login credentials. In order to clear these contents, you must do so from the browser.
- Upload functionality is a three step process:
 - First the application asks the server for an upload URL and an ID. In addition, one or more files are selected for upload. NOTE: The number of files that can be uploaded at any one time depends on the type of user account.
 - After choosing what files to upload, clicking submit will start the actual **Upload** process.
 - **Commit Upload** automatically starts after the files have been successfully uploaded. When the operation completes successfully, a response will appear containing the download URL for the recently uploaded batch.
- When downloading a file, the download feature of the browser is used. For instance, in the case of Firefox, the pop-up window with Save and Open radio buttons will appear. Because of this feature is built into the browser, no success or error messages will be returned in the log. Additional details are available in [Chapter 4.2 FileTransfer.php](#)

4. Library Reference

This section presents the method's signatures, documentation and samples of their usage. The methods are acting as wrappers for the calls mentioned in YSI API Developer's Guide.

4.1. Authentication.php

4.1.1. Login

This method is used to authenticate with YouSendIt and assigns the returned token to the *YouSendItDPI* instance for all future calls. It is a wrapper for the **GET Authentication Token** REST API described in the YSI API Developer's Guide.

NOTE: This method must return a valid authenticating token (not null) and no exceptions before you can call the rest of the API methods.

The method returns an *Auth* object with the following params:

string *Auth::*\$authToken

Signature:

```
* Gets user authentication token.
*
* @param email - required -
*     The email address of the user.
* @param password - required -
*     The password of the user.
* @return An Auth object. If the call is successful, the authentication token is set.
*     If unsuccessful, the object contains the error code and message thrown by
*     the server.
*/
```

```
public function login($email, $password)
```

Usage Sample:

```
<?php
require_once('Authentication.php');
require_once('RestTransport.php');
session_start();
$restTransportInstance = new RestTransport($host,$apikey);
$_SESSION['sTr'] = $restTransportInstance;

$authInstance = new Authentication($endpoint,$apikey, $restTransportInstance);
$auth = $authInstance->login($email,$password);

$errorStatus = $auth->getErrorStatus();
if (empty($errorStatus)){
    $token = $auth->getAuthToken();
    $_SESSION['authToken'] = $token;
}else{
    $code = $errorStatus->getCode();
    $message = $errorStatus->getMessage();
}
?>
```

The *Authentication* class has to be included in order to call the *login* method. If the call is successful, the error status is empty and the token is set. It may be useful to set the authorization token as a session parameter since it will be required for all the other method calls. Also, an instance of the class *RestTransport* is necessary for all calls. Therefore, this instance should also be kept as a session parameter.

4.2. UserAPIs.php

4.2.1. Create user

This method creates a new user account. This method is the wrapper for the **Create User** REST API described in the YSI API Developer's Guide. The user is sent an email with an activation link that they must click to activate their account before they can use the account.

NOTE: The newly created user account is a YouSendIt Lite account, which is a free account. Check the link at <http://www.yousendit.com/compare-plans> to see the storage limits and maximum upload size for the Lite users.

NOTE: This method is not implemented in the Sample app

The method returns a *User* object with the following attributes:

```
@var string $status
@var string $email
@var string $firstName
@var string $lastName
@var string $type
@var string $version
@var object $account
@var object $storage
```

Signature:

```
/**
 * Creates new user
 * @param email - required -
 *     The email address of the new user.
 *     Maximum 128 characters are allowed in the email address.
 * @param password - required -
 *     The password for the new user.
 *     Minimum 5 characters are required for the password.
 * @param firstName - optional -
 *     The first name of the user.
 *     Maximum 128 characters are allowed in the first name.
 * @param lastName - optional -
 *     The last name of the user.
 *     Maximum 128 characters are allowed in the last name.
 * @return The User object. If the operation was successful, the object
 *     contains the status, if not, the object contains the
 *     error code and message.
 */
public function createUser($email, $password, $firstName='', $lastName='')
```

Usage Sample:

```
<?php
require_once('UserAPIs.php');

$userAPI = new UserAPIs($endpoint,$apikey,$transport);
$user = $userAPI->createUser($email, $password, $firstName, $lastName);
$errorStatus = $user->getErrorStatus();
if (empty($errorStatus)){
    $status = $user->getStatus();
}else{
```



```

        $code = $errorStatus->getCode();
        $message = $errorStatus->getMessage();
    }?>

```

4.2.2. Get user info

This method returns information about a user. It is the wrapper for “Get User Info” API call. The method returns a *User* object with the following attributes:

```

@var string $status
@var string $email
@var string $firstName
@var string $lastName
@var string $type
@var string $version
@var object $account
@var object $storage

```

Signature:

```

* Get User Info
* @param email - required -
*     The email address of the user for which information is requested.
* @return The User object. If the operation was successful, the object contains
* the user info, if not, the object contains the error code and message.
public function getUserInfo($email)

```

Usage Sample:

```

<?php
require_once('UserAPIs.php');
session_start();

$userAPI = new UserAPIs($endpoint,$apikey,$transport);
$userAPI->setAuthToken($_SESSION['authToken']);
$user = $userAPI->getUserInfo($email);

$errorStatus = $user->getErrorStatus();
if (empty($errorStatus)){
    $firstName = $user->getFirstName();
    //...
}else{
    $code = $errorStatus->getCode();
    $message = $errorStatus->getMessage();
}
?>

```

4.2.3. Get policy info

This method returns all the user policy information. It is a wrapper for the **Get Policy Info** REST API described in the YSI API Developer’s Guide. This method returns a *Policy* object.

NOTE: This method is available only for Enterprise users. If the requesting user does not have Enterprise-level privileges, the YouSendIt server will return an error: “*Not an enterprise user*”

The method returns a *Policy* object with the following attributes:

```

@var string $fileExpiration
@var string $fileExpirationOverwrite
@var string $passwordProtect

```

```

@var string $passwordProtectOverwrite
@var string $returnReceipt
@var string $returnReceiptOverwrite
@var string $vri
@var string $vriOverwrite
@var string $vriShare
@var string $vriOverwriteShare
@var string array $blackListedDomains
@var string array $whiteListedDomains
@var string array $blackListedDomainsShare
@var string array $whiteListedDomainsShare

```

Signature:

```

/**
 * Gets policy info (for an enterprise user)
 * @return The Policy object. If the operation was successful, the object
 *         contains the status, if not, the object contains the error code and message.
 */
public function getPolicyInfo()

```

Usage Sample:

```

<?php
require_once('UserAPIs.php');
session_start();

$userAPI = new UserAPIs($endpoint,$apikey,$transport);
$userAPI->setAuthToken($_SESSION['authToken']);
$policy = $userAPI->getPolicyInfo();

$errorStatus = $policy->getErrorStatus();
if (empty($errorStatus)){
    $fileExp = $policy->getFileExpiration();
    //...
}else{
    $code = $errorStatus->getCode();
    $message = $errorStatus->getMessage();
}??>

```

4.3. SendReceive.php

4.3.1. Get upload URL for a new item

This method is used to initialize an upload. It retrieves the *uploadURL* list from the server for a new item batch of one or more files. This method is a wrapper for the **Get upload URL for a new Item** REST API described in the YSI API Developer's Guide. The ability to upload more than one file depends on the type of account associated with this user. Verify Recipient Identity, Return Receipt, and Password, are options which may or may not be available depending on the underlying account associated with this user. For corporate users, this API honors the policies set by account administrators. This method returns an object containing the upload URL for a new item.

The return object is *Upload*:

```
@var string $fileId
@var string $uploadUrl
```

Signature:

```
/**
 * Get Upload Urls
 * @param recipients - required -
 *     Comma separated email addresses of the recipients.
 *     Maximum 300 email addresses can be provided.
 *     The maximum length of an email address is 128 characters.
 * @param filecount - optional -
 *     The number of files to be sent. Maximum 20 files can be sent at a time.
 * @param subject - optional -
 *     The subject for the email notification. Maximum 200 characters are
 *     allowed in the subject.
 * @param message - optional -
 *     The message for the email body.
 *     Maximum 2000 characters are allowed in the message.
 * @param verifyIdentity - optional -
 *     If true, the recipients are required to login to access the files.
 * @param returnReceipt - optional -
 *     If true, an email notification is sent to the sender, when
 *     someone downloads the file.
 * @param password - optional -
 *     If set, the recipients are asked to enter the password before
 *     they can access the files.
 *     The minimum length of the password is 5 characters.
 * @return An Upload object with upload URLs or the error code and message
 *     thrown by the server.
 */
public function getUploadURLs($recipients, $filecount='', $subject='', $message='',
$verifyIdentity='', $returnReceipt='', $password='')
```

Usage Sample:

```
<?php
require_once('SendReceive.php');
session_start();

$sendReceiveAPI = new SendReceive($endpoint,$apikey,$transport);
$sendReceiveAPI->setAuthToken($_SESSION['authToken']);
$upload = $sendReceiveAPI->getUploadURLs($recipients);

$errorStatus = $upload->getErrorStatus();
```

```

if (empty($errorStatus)){
    $urls = $upload->getUploadUrl();
    //...
}else{
    $code = $errorStatus->getCode();
    $message = $errorStatus->getMessage();
}??>

```

4.3.2. Commit item send

This method implements the third, and final, step of the File Upload flow. It is a wrapper for the **Commit Item Send** REST API call described in the YSI API Developer's Guide. The ability to set expiration depends on the underlying account associated with this user. If the account type does not allow you to set the expiration, skip this parameter. It returns an object containing the download URL for the item.

The return object is *Commit*:

```
@var string $downloadUrl
```

Signature:

```

/**
 * Commit item send
 * @param itemId - required -
 *     The item id.
 * @param sendEmailNotifications - optional -
 *     If true, an email notification is sent to the sender and the recipients.
 *     If false, no email is sent.
 * @param expiration - optional -
 *     The expiration duration in minutes.
 *     A value of 0 indicates that the item never expires.
 * @return The Commit object containing the download URL(link to downloadpage)
 * or the error code and message returned by the server.
 */
public function commitSend($itemId,$sendEmailNotifications='', $expiration='')

```

Usage Sample:

```

<?php
require_once('SendReceive.php');
session_start();

$sendReceiveAPI = new SendReceive($endpoint,$apikey,$transport);
$sendReceiveAPI->setAuthToken($_SESSION['authToken']);
$commit = $sendReceiveAPI->commitSend($itemId);

$errorStatus = $commit->getErrorStatus();
if (empty($errorStatus)){
    $downloadUrl = $commit->getDownloadUrl();
}else{
    $code = $errorStatus->getCode();
    $message = $errorStatus->getMessage();
}??>

```

4.3.3. Get upload status

This method is used to monitor and display the progress of a file upload operation. It is a wrapper for the **Get upload status during the upload** REST API call described in the YSI API Developer's

Guide.

This method returns a *Status* object:

```
@var string $bytesReceived
@var string $status
```

Signature:

```
/**
 * Get upload status
 * @param uploadurl - required -
 *                     The upload url.
 * @return The Status object containing the number of bytes uploaded
 *         successfully or the error code and message thrown by the server.
 */
public function getUploadStatus($uploadurl)
```

Usage Sample:

```
<?php
require_once('SendReceive.php');
session_start();

$sendReceiveAPI = new SendReceive($endpoint,$apikey,$transport);
$sendReceiveAPI->setAuthToken($_SESSION['authToken']);
$status = $sendReceiveAPI->getUploadStatus($uploadurl);

$errorStatus = $status->getErrorStatus();
if (empty($errorStatus)){
    $bytes = $status->getBytesReceived();
    //...
}else{
    $code = $errorStatus->getCode();
    $message = $errorStatus->getMessage();
}
?>
```

4.3.4. Get list of items in user's Inbox and/or Sent items

The method is used to get a list of items in the user's Inbox or Sent Items folder. It is a wrapper for the **Get list of items in user's Inbox or Sent Items** REST API call described in the YSI API Developer's Guide.

This method returns an object containing the list of *Items*:

```
@var object array $items
@var string $create
@var string $expiration
@var string $file
@var string $id
@var string $message
@var string $recipients
@var string $subject
@var object array $files
```

Signature:

```
/**
 * Get List of Items
 * @param sentItems - optional -
 * If true, the API returns the number of items in user's Sent Items only
 * otherwise it returns the number of items in user's Inbox and Sent Items.
 * @param filter - optional -
```

```

*         all: return all items
*         expired: returns all expired items
*         unexpired: returns only unexpired items
* @param includeFileInfo - optional -
*         If true, it returns detailed file info.
* @param includetracking - optional -
*         If true, it returns the tracking information about the item.
* @param page - optional -
*         The API supports pagination to return information in chunks.
* @param pageLength - optional -
*         The number of items to return in a page. Maximum value is 100.
* @return A response with Items object containing 0 or more item elements.
* Each item element has details like itemId, expiration date, file details
* etc. or the error code and message returned by the server.
*/
public function getItemList($sentItems='', $filter='', $includeFileInfo='',
$includeTracking='', $page='', $pageLength='')

```

Usage Sample:

```

<?php
require_once('SendReceive.php');
session_start();

$sendReceiveAPI = new SendReceive($endpoint,$apikey,$transport);
$sendReceiveAPI->setAuthToken($_SESSION['authToken']);
$items = $sendReceiveAPI->getItemList();

$errorStatus = $items->getErrorStatus();
if (empty($errorStatus)){
    $itemArray = $items->getItems();
    //...
}else{
    $code = $errorStatus->getCode();
    $message = $errorStatus->getMessage();
}
?>

```

4.3.5. Get item count

This method returns the item count from Sent and Inbox items respectively. The method is the wrapper for the **Get item count** REST API call described in the YSI API Developer's Guide. This method returns an object containing the number of items in Inbox or Sent Items.

The return method is a *Count* object:

```
@var string $itemCount
```

Signature:

```

/**
 * Get Item Count
 * @param sentItems - optional -
 * If true, the API returns the number of items in user's Sent Items
 * otherwise it returns the number of items in user's Inbox and Sent Items.
 * @param filter - optional -
 *         all: return all items
 *         expired: returns all expired items
 *         unexpired: returns only unexpired items
 * @return The Count object containing the number of items or the error code

```

```

        * and message returned by the server.
    */
    public function getItemsCount($sentItems='', $filter='')

```

Usage Sample:

```

<?php
require_once('SendReceive.php');
session_start();

$sendReceiveAPI = new SendReceive($endpoint, $apikey, $transport);
$sendReceiveAPI->setAuthToken($_SESSION['authToken']);
$count = $sendReceiveAPI->getItemsCount();

$errorStatus = $count->getErrorStatus();
if (empty($errorStatus)){
    $itemCount = $count->getItemCount();
    //...
}else{
    $code = $errorStatus->getCode();
    $message = $errorStatus->getMessage();
}
?>

```

4.3.6. Get item info

This method returns information about a particular *item* given its *itemId*. It is a wrapper for the **Get item info** REST API call described in the YSI API Developer's Guide. It returns all the information available for a particular item given its ItemID. The API returns the tracking element in the XML response only if tracking info is available for the item. Tracking is not available if the user is a Lite user or if user is looking at the Inbox items.

This method returns an *Item* object:

```

@var string $create
@var string $expiration
@var string $file
@var string $id
@var string $message
@var string $recipients
@var string $subject
@var object array $files

```

Signature:

```

/**
 * Get Item Info
 * @param itemId - required -
 *                Item id
 * @return An Item object with details like itemId, expiration date, file details
 *         etc.
 *         * or the error code and message returned by the server.
 */
    public function getItemInfo($itemId)

```

Usage Sample:

```

<?php
require_once('SendReceive.php');
session_start();

```

```

$sendReceiveAPI = new SendReceive($endpoint,$apikey,$transport);
$sendReceiveAPI->setAuthToken($_SESSION['authToken']);
$item = $sendReceiveAPI->getItemInfo($itemId);

$errorStatus = $item->getErrorStatus();
if (empty($errorStatus)){
    $create = $item->getCreate();
    //...
}else{
    $code = $errorStatus->getCode();
    $message = $errorStatus->getMessage();
}??>

```

4.3.7. Change expiration of an item in user's Sent Items

This method changes the expiration date for a particular *item* given its *itemId*. This method is a wrapper for the **Change expiration of an item in Sent Items** REST API call described in the YSI API Developer's Guide. This method returns an object containing the new expiration date.

Returns an *Expiration* object:

@var string \$status

Signature:

```

/**
 * Change Item Expiration
 * @param itemId - required -
 *     Item Id.
 * @param expiration - required -
 *     The expiration duration in minutes. A value of 0 indicates that the file
 *     never expires.
 * @return An Expiration object with expiration date as status or the error code and
 *     message returned by the server.
 */
public function changeExpiration($itemId,$expiration)

```

Usage Sample:

```

<?php
require_once('SendReceive.php');
session_start();

$sendReceiveAPI = new SendReceive($endpoint,$apikey,$transport);
$sendReceiveAPI->setAuthToken($_SESSION['authToken']);
$expiration = $sendReceiveAPI->changeExpiration($itemId, $expiration);

$errorStatus = $expiration->getErrorStatus();
if (empty($errorStatus)){
    $expStatus = $expiration->getStatus();
    //...
}else{
    $code = $errorStatus->getCode();
    $message = $errorStatus->getMessage();
}??>

```


4.3.8. Delete an item from user's Inbox or Sent Items

This method deletes an *item* given its *itemId*. This method is a wrapper for the **Delete an Item** REST API call described in the YSI API Developer's Guide. This method returns an object containing the status of the delete operation.

Returns a *Delete* object:

```
@var string $status
```

Signature:

```
/**
 * Delete item
 * @param itemId - required -
 *               Item id
 * @return A Delete object with status value OK
 * or the error code and message returned by the server.
 */
public function deleteItem($itemId)
```

Usage Sample:

```
<?php
require_once('SendReceive.php');
session_start();

$sendReceiveAPI = new SendReceive($endpoint,$apikey,$transport);
$sendReceiveAPI->setAuthToken($_SESSION['authToken']);
$delete = $sendReceiveAPI->deleteItem($itemId);

$errorStatus = $delete->getErrorStatus();
if (empty($errorStatus)){
    $delStatus = $delete->getStatus();
    //...
}else{
    $code = $errorStatus->getCode();
    $message = $errorStatus->getMessage();
}??>
```

4.4. StorageAPIs.php

4.4.1. Create new folder

This method creates a new folder given a folder *name* and a *parentId*. This method is a wrapper for the **Create new folder** REST API described in the YSI API Developer's Guide. The method returns an object containing information about the newly created folder.

Returns a *Folder* object:

```
@var string $revision
@var string $id
@var string $createdOn
@var string $fileCount
@var object array $files
@var string $folderCount
@var object array $folders
@var string $name
@var string $parentId
@var string $readable
@var string $size
@var string $type
@var string $updatedOn
@var string $writeable
@var string $currentUsage
@var string $storageQuota
@var string $ownedByUser
@var string $deletedStatus
@var string $folderId
@var string $folderType
@var string $lastUpdatedOn
@var object array $permissions
@var string $shareInvitationPending
```

Signature:

```
/**
 * Create New Folder
 * @param name - required -
 *   The name of the folder.
 * @param parentId - optional -
 *   The id of the folder. A value of 0 indicates the root folder. Default=0
 * @return The Folder object. If the operation was successful, the object
 *   contains the folder info, if not, the object contains the
 *   error code and message
 */
public function createFolder($name, $parentid='')
```

Usage Sample:

```
<?php
require_once('StorageAPIs.php');
session_start();

$storageAPI = new StorageAPIs($endpoint,$apikey,$transport);
$storageAPI->setAuthToken($_SESSION['authToken']);
$folder = $storageAPI->createFolder($name);

$errorStatus = $folder->getErrorStatus();
```

```

if (empty($errorStatus)){
    $folderId = $folder->getId();
    //...
}else{
    $code = $errorStatus->getCode();
    $message = $errorStatus->getMessage();
}??>

```

4.4.2. Get folder info

This method returns all the information for a particular *Folder* object given its *folderId*. 0 (zero) is the *folderId* for the root folder. This method is a wrapper for the **Get folder info** REST API described in the YSI API Developer's Guide. The method returns an object containing information about a folder (Please see section [4.4.1 Create new folder](#))

Signature:

```

/**
 * Get Folder Info
 * @param folderId - required -
 *     The id of the folder.
 * @param includeFiles - optional -
 *     Default = true. If true, the API returns info about all the files
 *     in the specified folder, at this folder level only.
 * @param includeFolders - optional -
 *     Default = true. If true, the API returns info about all the sub
 *     folders, at this folder level only
 * @return The Folder object. If the operation was successful, the object
 *     contains the folder info, if not, the object
 *     contains the error code and message.
 */
public function getFolderInfo($folderId, $includeFiles='', $includeFolders='')

```

Usage Sample:

```

<?php
require_once('StorageAPIs.php');
session_start();

$storageAPI = new StorageAPIs($endpoint,$apikey,$transport);
$storageAPI->setAuthToken($_SESSION['authToken']);
$folder = $storageAPI->getFolderInfo($folderId);

$errorStatus = $folder->getErrorStatus();
if (empty($errorStatus)){
    $folderName = $folder->getName();
    //...
}else{
    $code = $errorStatus->getCode();
    $message = $errorStatus->getMessage();
}??>

```

4.4.3. Get file info

This method returns all the information for a particular *File* object given its *fileId*. This method acts as a wrapper for the **Get file info** REST API described in the YSI API Developer's Guide. This method returns an object containing information about a file.

The method returns a *File* object:

```
@var string $revision
@var string $id
@var string $createdOn
@var string $name
@var string $parentId
@var string $size
@var string $status
@var string $clickableDownloadUrl
@var string $ownedByStorage
@var string $downloadUrl
@var string $downloads
@var string $passwordProtect
@var string $returnReceipt
@var string $verifyIdentity
@var object array $tracking
@var string $deleteStatus
@var string $folderId
@var string $fileId
@var string $size
@var string $fileName
@var object array $revisions
@var string $revisionCount
```

Signature:

```
/**
 * Get File Info
 * @param fileId - required -
 *                 The id of the file.
 * @return The File object. If the operation was successful, the object contains
 *         the file info,if not,the Folder object containing the error code and message.
 */
public function getFileInfo($fileId)
```

Usage Sample:

```
<?php
require_once('StorageAPIs.php');
session_start();

$storageAPI = new StorageAPIs($endpoint,$apikey,$transport);
$storageAPI->setAuthToken($_SESSION['authToken']);
$file = $storageAPI->getFileInfo($fileId);

$errorStatus = $file->getErrorStatus();
if (empty($errorStatus)){
    $fileName = $file->getName();
    //...
}else{
    $code = $errorStatus->getCode();
    $message = $errorStatus->getMessage();
}??>
```

4.4.4. Initialize file upload to a folder

This method initializes the file upload flow. The method returns the upload details from the server used in the *Upload* and *CommitUpload* methods. It is a wrapper for the **Initialize file upload to a folder** REST API described in the YSI API Developer's Guide. This method returns an object

containing the file id and the upload URL.

The return object is *Upload*:

```
@var string $fileId
@var string $uploadUrl
```

Signature:

```
/**
 * Initiate file upload to a folder
 * @return The Upload object with fileId and uploadUrl . If the operation was
 *         successful, the object contains the fileId and uploadUrl,
 *         if not, the Upload object contains the error code and message.
 */
public function initUpload()
```

Usage Sample:

```
<?php
require_once('StorageAPIs.php');
session_start();

$storageAPI = new StorageAPIs($endpoint,$apikey,$transport);
$storageAPI->setAuthToken($_SESSION['authToken']);
$upload = $storageAPI->initUpload();

$errorStatus = $upload->getErrorStatus();
if (empty($errorStatus)){
    $uploadUrl = $upload->getUploadUrl();
    //...
}else{
    $code = $errorStatus->getCode();
    $message = $errorStatus->getMessage();
}??
```

4.4.5. Commit file upload to folder

This method performs the third and final step in the file upload flow. It is a wrapper for the **Commit file upload to a folder** REST API described in the YSI API Developer's Guide. This method returns an object containing information about the uploaded file. The return value is a *File* object(please see section [4.4.3 Get file info](#))

Signature:

```
/**
 * Commit file upload to a folder
 * @param fileId - required -
 *         The id of the file.
 * @param parentId - optional -
 *         The id of the folder, where the file will be saved. A value of 0
 *         indicates root folder.
 * @return The File object. If the operation was successful, the object contains
 *         the file info, if not, the object contains the error code and message.
 */
public function commitFileUpload($fileId,$parentId='')
```

Usage Sample:

```
<?php
require_once('StorageAPIs.php');
```

```

session_start();

$storageAPI = new StorageAPIs($endpoint,$apikey,$transport);
$storageAPI->setAuthToken($_SESSION['authToken']);
$file = $storageAPI->commitFileUpload($fileId);

$errorStatus = $file->getErrorStatus();
if (empty($errorStatus)){
    $fileSize = $file->getSize();
    //...
}else{
    $code = $errorStatus->getCode();
    $message = $errorStatus->getMessage();
}??>

```

4.4.6. Rename a file in a folder

This method changes the name of an existing file. This method is a wrapper for the *Rename File* REST API described in the YSI API Developer's Guide. The method returns an object containing the status of the operation.

The return object is *Rename*:

```
@var string $status
```

Signature:

```

/**
 * Rename a file
 * @param name - required -
 *             The new name for the file.
 *
 * @param fileId - required -
 *             The file id for the file to be renamed.
 *
 * @return The Rename object containing the status of the operation.
 */
public function renameFile($name,$fileId)

```

Usage Sample:

```

<?php
require_once('StorageAPIs.php');
session_start();

$storageAPI = new StorageAPIs($endpoint,$apikey,$transport);
$storageAPI->setAuthToken($_SESSION['authToken']);
$rename = $storageAPI->renameFile($name, $fileId);

$errorStatus = $rename->getErrorStatus();
if (empty($errorStatus)){
    $renameStatus = $rename->getStatus();
    //...
}else{
    $code = $errorStatus->getCode();
    $message = $errorStatus->getMessage();
}??>

```

4.4.7. Move a file to another folder

This method changes the location path of one file by assigning it another *parentId*. This method is a wrapper for the **Move a file in a Folder** REST API described in the YSI API Developer's Guide. This method returns an object containing the status of the operation.

The return object is *Move*:

@var string \$status

Signature:

```
/**
 * Move a file
 * @param fileId - required -
 *                 The file id for the file to be moved.
 *
 * @param parentId - required -
 *                 Id of the parent where the file will be moved to.
 * @return The Move object containing the status of the operation.
 */
public function moveFile($fileId,$parentId)
```

Usage Sample:

```
<?php
require_once('StorageAPIs.php');
session_start();

$storageAPI = new StorageAPIs($endpoint,$apikey,$transport);
$storageAPI->setAuthToken($_SESSION['authToken']);
$move = $storageAPI->moveFile($fileId, $parentId);

$errorStatus = $move->getErrorStatus();
if (empty($errorStatus)){
    $moveStatus = $move->getStatus();
    //...
}else{
    $code = $errorStatus->getCode();
    $message = $errorStatus->getMessage();
}??>
```

4.4.8. Delete a file from a folder

This method marks a file as deleted given its *fileId*. This method is a wrapper for the **Delete File** REST API described in the YSI API Developer's Guide. This method returns an object containing the status of the operation.

The return object is *Delete*:

@var string \$status

Signature:

```
/**
 * Delete a file
 * @param fileId - required -
 *                 The file id for the file to be moved.
 *
 * @return The Delete object containing the status of the operation.
 */
```

```
public function deleteFile($fileId)
```

Usage Sample:

```
<?php
require_once('StorageAPIs.php');
session_start();

$storageAPI = new StorageAPIs($endpoint,$apikey,$transport);
$storageAPI->setAuthToken($_SESSION['authToken']);
$delete = $storageAPI->deleteFile($fileId);

$errorStatus = $delete->getErrorStatus();
if (empty($errorStatus)){
    $deleteStatus = $delete->getStatus();
    //...
}else{
    $code = $errorStatus->getCode();
    $message = $errorStatus->getMessage();
}??>
```

4.4.9. Rename folder

This method changes the name of an existing folder. The user must have write permissions for the particular folder. This method is a wrapper for the **Rename folder** REST API described in the YSI API Developer's Guide. This method returns an object containing the status of the operation.

The return object is *Rename*:

```
@var string $status
```

Signature:

```
/**
 * Rename Folder
 * @param name - required -
 *             The new name of the folder.
 * @param folderId - required -
 *             The id of the folder.
 * @return The Rename object which contains the status of the operation.
 */
public function renameFolder($name, $folderId)
```

Usage Sample:

```
<?php
require_once('StorageAPIs.php');
session_start();

$storageAPI = new StorageAPIs($endpoint,$apikey,$transport);
$storageAPI->setAuthToken($_SESSION['authToken']);
$rename = $storageAPI->renameFolder($name, $folderId);

$errorStatus = $rename->getErrorStatus();
if (empty($errorStatus)){
    $renameStatus = $rename->getStatus();
    //...
}else{
    $code = $errorStatus->getCode();
```



```

        $message = $errorStatus->getMessage();
    }?>

```

4.4.10. Move folder

This method changes the location path of one folder by assigning it another *parentId*. This method is a wrapper for the **Move Folder** REST API described in the YSI API Developer's Guide. This method returns an object containing the status of the operation.

The return object is *Move*:

```
@var string $status
```

Signature:

```

/**
 * Move Folder
 * @param folderId - required -
 *                   The id of the folder.
 * @param parentId - required -
 *                   The id of the parent where the file should be moved.
 * @return The Move object which contains the status of the operation.
 */
public function moveFolder($folderId, $parentId)

```

Usage Sample:

```

<?php
require_once('StorageAPIs.php');
session_start();

$storageAPI = new StorageAPIs($endpoint,$apikey,$transport);
$storageAPI->setAuthToken($_SESSION['authToken']);
$move = $storageAPI->moveFolder($folderId, $parentId);

$errorStatus = $move->getErrorStatus();
if (empty($errorStatus)){
    $moveStatus = $move->getStatus();
    //...
}else{
    $code = $errorStatus->getCode();
    $message = $errorStatus->getMessage();
}
?>

```

4.4.11. Delete a folder

This method marks a folder as deleted given its *folderId*. This method is a wrapper for the **Delete Folder** REST API described in the YSI API Developer's Guide. This method returns an object containing the status of the operation.

The return object is *Delete*:

```
@var string $status
```

Signature:

```

/**
 * Delete Folder
 * @param folderId - required -
 *                   The id of the folder.
 * @return The Delete object which contains the status of the operation.

```

```

        *
    */
    public function deleteFolder($folderId)

```

Usage Sample:

```

<?php
require_once('StorageAPIs.php');
session_start();

$storageAPI = new StorageAPIs($endpoint,$apikey,$transport);
$storageAPI->setAuthToken($_SESSION['authToken']);
$delete = $storageAPI->deleteFolder($folderId);

$errorStatus = $delete->getErrorStatus();
if (empty($errorStatus)){
    $deleteStatus = $delete->getStatus();
    //...
}else{
    $code = $errorStatus->getCode();
    $message = $errorStatus->getMessage();
}
?>

```

4.4.12. Get file revisions

This method returns the revision history of a particular file given its *fileId*. The method is a wrapper for the **Get file revisions** REST API described in the YSI API Developer's Guide. This method returns an object containing a file's information and revisions (return object described in section [4.4.3 Get file info](#)).

Signature:

```

/**
 * Get File Revisions
 * @param fileId - required -
 *                 The id of the file.
 *
 * @return  A File object is return with all the revisions
 *          or the error code and message returned by the server.
 */
    public function getFileRevisions($fileId)

```

Usage Sample:

```

<?php
require_once('StorageAPIs.php');
session_start();

$storageAPI = new StorageAPIs($endpoint,$apikey,$transport);
$storageAPI->setAuthToken($_SESSION['authToken']);
$file = $storageAPI->getFileRevisions($fileId);

$errorStatus = $file->getErrorStatus();
if (empty($errorStatus)){
    $fileName = $file->getName();
    //...
}else{
    $code = $errorStatus->getCode();
    $message = $errorStatus->getMessage();
}

```

```
}?>
```

4.4.13. Get Storage Revisions

This method returns a list of all added, deleted, moved, renamed folders and added, updated, deleted, renamed, moved files. This method is a wrapper for the **Get Storage Revisions** REST API described in the YSI API Developer's Guide. This method returns an object containing the list of added, deleted, moved, renamed folders and added, updated, deleted, renamed, moved files.

The return object is *StorageChanges*:

```
@var string $currentUsage
@var object array $files
@var object array $folders
@var string $revision
@var string $revisionCount
@var string $statusDisplay
@var string $storageQuota
@var string $workspaceId
```

Signature:

```
/**
 * Get Storage Revisions
 * @param fromRevision - optional -
 *                      Revision Number
 * @return A StorageChanges object is returned with a list of all the file and
 *                      folder revisions.
 */
public function getStorageRevisions($fromRevision='')
```

Usage Sample:

```
<?php
require_once('StorageAPIs.php');
session_start();

$storageAPI = new StorageAPIs($endpoint,$apikey,$transport);
$storageAPI->setAuthToken($_SESSION['authToken']);
$changes = $storageAPI->getStorageRevisions($fromRevision);

$errorStatus = $changes->getErrorStatus();
if (empty($errorStatus)){
    $count = $changes->getRevisionCount();
    //...
}else{
    $code = $errorStatus->getCode();
    $message = $errorStatus->getMessage();
}?>
```

4.5. FolderShareSendAPIs.php

4.5.1. Share a folder

This method allows the user to share a specific folder, given its *folderId*, with other users by providing a comma-separated list of e-mail addresses. This method is a wrapper for the **Share a folder** REST API described in the YSI API Developer's Guide. Notifications will be sent to all of the e-mail addresses provided as input. This method returns an object containing the status of the operation.

Returns a *Share* object:

```
@var string $status
@var object array $members
```

Signature:

```
/**
 * Share folder with other users
 * @param folderId - required -
 *     Id of the folder to be shared.
 * @param email - required -
 *     Comma separated email addresses
 *     of the users to share folders with.
 * @param message - optional -
 *     The message that will go in the invitation email.
 *     Optional parameter. Default: empty message.
 * @param permission - optional -
 *     If set to read, the user cannot update the content of the folder.
 *     If set to true, the user can update the content of the folder.
 *     Optional parameter. Default: 'read'.
 * @return The Share object containing the status of the operation.
 */
public function shareFolder($folderId, $email, $message='', $permission='')
```

Usage Sample:

```
<?php
require_once('FolderShareSendAPIs.php');
session_start();

$folderShareSendAPI = new FolderShareSendAPIs($endpoint,$apikey,$transport);
$folderShareSendAPI->setAuthToken($_SESSION['authToken']);
$share = $folderShareSendAPI->shareFolder($folderId, $email);

$errorStatus = $share->getErrorStatus();
if (empty($errorStatus)){
    $shareStatus = $share->getStatus();
    //...
}else{
    $code = $errorStatus->getCode();
    $message = $errorStatus->getMessage();
}>>
```

4.5.2. Get share info

This method returns all the available share information for a particular folder given its *folderId*. The method is a wrapper for the **Get share info** REST API described in the YSI API Developer's Guide.

The method returns an object containing information about all the shares of a folder.

Returns a *Share* object:

@var string \$status

@var object array \$members

Signature:

```
/**
 * Get Share Info - get the share info of a folder
 * @param folderId - required -
 *                   Id of the folder.
 * @return The Share object containing the share folder info or the error status
 *         of the operation
 */
public function getShareInfo($folderId)
```

Usage Sample:

```
<?php
require_once('FolderShareSendAPIs.php');
session_start();

$folderShareSendAPI = new FolderShareSendAPIs($endpoint,$apikey,$transport);
$folderShareSendAPI->setAuthToken($_SESSION['authToken']);
$share = $folderShareSendAPI->getShareInfo($folderId);

$errorStatus = $share->getErrorStatus();
if (empty($errorStatus)){
    $sharedMembersArray = $share->getMembers();
    //...
}else{
    $code = $errorStatus->getCode();
    $message = $errorStatus->getMessage();
}
?>
```

4.5.3. Unshare a folder

This method deletes all the share references of an e-mail address for a particular folder given its *folderId* and the respective e-mail address. This method is a wrapper for the **Unshare folder** REST API described in the YSI API Developer's Guide.

NOTE: In order to unshare a folder already shared with a particular user, that user needs to have previously accepted the share invitation sent by the YouSendIt server. If this is not the case, the server will return the following message: "*email@domain.com has not accepted the invitation*". This method returns an object containing the status of the operation.

The return object is *Delete*:

@var string \$status

Signature:

```
/**
 * Unshare Folder
 * @param folderId - required -
```

```

        *           Id of the folder.
        * @param email - required -
        *           The email address that needs to be removed from the share
        * @return The Delete object containing the status of the operation.
    */
    public function unshareFolder($folderId,$email)

```

Usage Sample:

```

<?php
require_once('FolderShareSendAPIs.php');
session_start();

$folderShareSendAPI = new FolderShareSendAPIs($endpoint,$apikey,$transport);
$folderShareSendAPI->setAuthToken($_SESSION['authToken']);
$unshare = $folderShareSendAPI->unshareFolder($folderId, $email);

$errorStatus = $unshare->getErrorStatus();
if (empty($errorStatus)){
    $unshareStatus = $unshare->getStatus();
    //...
}else{
    $code = $errorStatus->getCode();
    $message = $errorStatus->getMessage();
}
?>

```

4.5.4. Send a file from a folder

This method sends a request to the YouSendIt Storage Server to share a particular file with other recipients given the *fileId* and a comma-separated list of e-mail addresses. This method is a wrapper for the **Send a file from folder** REST API described in the YSI API Developer's Guide. This method returns an object containing the status of the operation (please see section [4.4.3 –Get file id](#) for full object description)

Signature:

```

/**
    * Send a file from the folder
    * @param fileId - required -
    *           Id of the file to be sent.
    * @param recipients - required -
    *           Comma separated email addresses
    *           of the users to whom the file link will be sent.
    * @return The File object containing the status of the operation.
    */
    public function sendFile($fileId,$recipients)

```

Usage Sample:

```

<?php
require_once('FolderShareSendAPIs.php');
session_start();

$folderShareSendAPI = new FolderShareSendAPIs($endpoint,$apikey,$transport);
$folderShareSendAPI->setAuthToken($_SESSION['authToken']);
$fileSend = $folderShareSendAPI->sendFile($fileId, $recipients);

$errorStatus = $fileSend->getErrorStatus();
if (empty($errorStatus)){

```

```
        $sendStatus = $fileSend->getStatus();  
        //...  
    }else{  
        $code = $errorStatus->getCode();  
        $message = $errorStatus->getMessage();  
    }?>
```

4.6. FileTransfer.php

4.6.1. Upload File

This method interacts with the YouSendIt storage servers based on the *itemId/fileId* and the *uploadURL* obtained from the PrepareSend/PrepareFileSend action. This method is a wrapper for the **Upload a file** REST API described in the YSI API Developer's Guide. This method is the second step in the upload process. It uses the upload URL returned by the **Init Upload / Get Upload URLs** method and uploads the file from the PHP server to the upload URL. The third step from the upload process is the execution of the **Commit File Upload / Commit Send** method.

Returned object is *YSIResponse*:

```
@var string $ufid
@var string $uploadStatus
```

Signature:

```
/**
 * Upload a File
 * @param upurl - required -
 *             The upload URL.
 * @param bid - required -
 *             The item Id returned in the same call with the upload URL.
 * @param fname - required -
 *             The name of the file to be uploaded. (path+fileName+fileExtension).
 * @return The YsiResponse object containing the upload status.
 */
public function uploadaFile($upurl,$bid,$fname)
```

Usage Sample:

```
<?php
require_once('./FileTransfer.php');
require_once('./include/ResponseEntities.php');

$ft = new FileTransfer($endpoint,$apikey,$useragent);
$ft->setAuthToken($authtoken);
$fname = $dir.$name;
$uploadObject = $ft->uploadaFile($uploadurl,$bid,$fname);
$err = $uploadObject->getYsiErrorStatus();
if (!empty($err)){
    $response = $err;
}else{
    $elup['ufid'] = $uploadObject->getUfid();
    $elup['ufid'] = $uploadObject->getUploadStatus();
    $response = $elup;
}??>
```

4.6.2. Download File

This method downloads a file inside the temporary download directory from the PHP server and returns the file name. The file name will be used in the application to search the temporary download directory for the downloaded file and push that file to the client. The method is using the entire server response (headers and body).

Signature:

```
/**
 * Download a File
```



```

* @param urlid - required -
*             The download URL.
* @param dir - required -
*             The download folder (path+folderName).
* @param password - optional -
*             If a password has been set, the recipients are asked to enter
*             the password before they can access the files. The minimum
*             length of the password is 5 characters.
* @return The status "fail" if the download has failed or the downloaded
*             file name if the download has succeeded.
*/
public function downloadaFile($urlid,$dir,$password='')

```

Usage Sample:

```

<?php
require_once('./FileTransfer.php');

$ft = new FileTransfer($endpoint,$apikey,$useragent);
$ft->setAuthToken($authtoken);
$dir = './downloads/';
if (!is_dir($dir)) {
    mkdir($dir);
}
$response = $ft->downloadaFile($downloadurl,$dir,$password);
if ($response!='fail'){
    $d = $response; //The name of the file downloaded on the PHP server
    $c = $dir.$response;
    include_once('./download.php');
}
else die('The download has failed.');
```

4.6.3. Download File Without Saving on the PHP server

This method returns the real download URL. The real download URL is extracted from the **Location** header present inside the redirect response. The real download URL will be used inside the application to redirect the user and download the file without being saved on the PHP server.

Signature:

```

/**
* Download a File Without Saving
* @param urlid - required -
*             The download URL.
* @param password - optional -
*             If a password has been set, the recipients are asked to enter the
*             password before they can access the files.
*             The minimum length of the password is 5 characters.
* @return The status "fail" if the download has failed or the actual
*             download URL if the download has succeeded.
*/
public function downloadaFileWS($urlid,$password='')

```

Usage Sample:

```

<?php
require_once('./FileTransfer.php');

$ft = new FileTransfer($endpoint,$apikey,$useragent);
$ft->setAuthToken($authtoken);

```

```

if (isset($password){
    $response = $ft->downloadaFileWS($downloadurl,$password);
}
else {
    $response = $ft->downloadaFileWS($downloadurl);
}
if ($response !== 'fail'){
    header("Location: $response");
}
else{
    header("Location: ./dwnl.php?err=1");
}??>

```

4.6.4. Download File with Resume Support

This method downloads or resumes a file download using the temporary download directory from the PHP server and returns the file name. The file name will be used inside the application to search the file inside the temporary download directory and push it to the client. Not using a **range** value will result in saving the entire file. The method is using only the body from the server response.

Signature:

```

/**
 * Resume the Download of a File
 * @param urld - required -
 *             The download URL.
 * @param dir - required -
 *             The download folder (path+folderName).
 * @param password - optional -
 *             If a password has been set, the recipients are asked to enter the
 *             password before they can access the files.
 *             The minimum length of the password is 5 characters.
 * @param range - optional -
 *             Only the specified range of bytes is downloaded and appended to
 *             the existing file if a range has been set. If no range has been
 *             set the entire file is downloaded and saved inside the download
 *             directory (the value from $dir). Range value example: '0-100'.
 * @return The status "Fail" if the download has failed or the downloaded
 *         file name if the download has succeeded.
 */
public function downloadaFileResume($urld,$dir,$password='', $range='')

```

Usage Sample:

```

<?php
require_once('./FileTransfer.php');

$a = new FileTransfer($endpoint,$apikey,$useragent);
$a->setAuthToken($authtoken);
$response = $a->downloadaFileResume($downloadurl,$dir,$password,$range); //Range
value example: '0-100'
if ($response !== 'fail'){
    return $response;
}
else{
    die('The download has failed.');
```